QATIP Intermediate Azure Lab 03b Azure Key Vault

Contents

Lab Objectives 1
Teaching Points 1
Before you begin 2
Step 1: Administrators set up Key Vault and SP 2
Step 2: Developers obtain SP credentials from Key Vault 4
Verifying the SP account 6
Lab clean-up 7

Lab Objectives

This lab provides guidance and instructions on setting up an Azure Key Vault and a Service Principal (SP) for securely managing Terraform authentication when running Terraform from an external device. This setup ensures that multiple developers can authenticate Terraform dynamically without needing to store or share sensitive credentials directly.

Teaching Points

Step 1 These tasks are typically handled by the **Security Team** to establish a secure authentication mechanism ensuring that only authorized developers can retrieve the necessary service principal credentials from Azure Key Vault.

Step 2 Performed by developers who have been granted the required **Get and List secret permissions** to retrieve authentication credentials used run Terraform commands.

Developers **do not have access to modify secrets** but can retrieve them for authentication.

Before you begin

Ensure you have completed Lab0 before attempting this lab.

In the IDE terminal pane, enter the following command...

cd c:\azure-tf-int\labs\03b

This shifts your current working directory to labs\03b. Ensure all commands are executed in this directory

Close any open files and use the Explorer pane to navigate to and open the **labs\03b** folder.

Step 1: Administrators set up Key Vault and SP

Examine the PowerShell script **setup-terraform-keyvault.ps1**

This script simulates tasks commonly performed by platform engineering or security teams responsible for securely provisioning Terraform authentication components.

Stage 1 - Define Lab Variables

The script begins by defining reusable variables including the resource group, Azure region, Key Vault name, subscription ID, and Service Principal name.
These values are intentionally parameterised to avoid hardcoded environment-specific values.

Stage 2 - Define Helper Functions

Reusable helper functions provide consistent Azure CLI execution, centralised error handling, and idempotent RBAC assignment logic. This mirrors enterprise operational scripting practices.

Stage 3 - Authenticate to Azure and Select Subscription

The script authenticates interactively using: az login
The script then explicitly selects the required Azure subscription to ensure all resources are deployed into the correct subscription boundary.

Stage 4 - Create or Reuse the Resource Group

The script checks whether the resource group already exists before attempting creation. This introduces the concept of idempotent automation where scripts can be safely re-run without creating duplicate resources.

Stage 5 - Create or Reuse the Azure Key Vault

The script checks whether the Key Vault already exists and creates it if required. The Key Vault is configured using Azure RBAC authorization mode rather than legacy vault access policies.

Stage 6 - Create or Reuse the Terraform Service Principal

The script creates a Service Principal used by Terraform for non-interactive authentication. If the Service Principal already exists, the script rotates the client secret instead of creating a duplicate identity.

Stage 7 - Configure Azure RBAC Permissions

The script assigns Azure RBAC roles to both the signed-in user and the Terraform Service Principal. The signed-in user receives Key Vault Secrets Officer permissions while the Terraform Service Principal receives Key Vault Secrets User permissions.

Stage 8 - Store Terraform Credentials as Key Vault Secrets

The following values are stored securely within Azure Key Vault:

- Terraform-Client-ID
- Terraform-Client-Secret
- Terraform-Tenant-ID
- Terraform-Subscription-ID

Terraform later consumes these values dynamically during authentication.

Stage 9 - Persist Lab Configuration Locally for Cleanup

The script stores deployment metadata locally to support automated cleanup operations later in the lab.

Stage 10 - Completion Confirmation

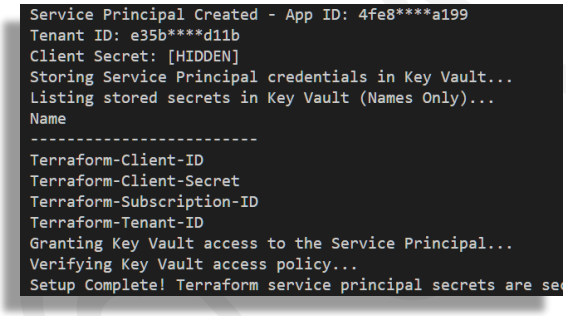
The script verifies successful completion by listing the stored secrets within Azure Key Vault.

Actions

1. Update **setup-terraform-keyvault.ps1** with a Vault name of your choice. This must be unique and can be up to 24 character long. It is suggested you use "**vault-**" followed by a portion of your subscription_id. Make a note of the name chosen.
2. Denmark East is specified as the location. If this region was unavailable for use when you completed Lab3a, then update line 7 with the alternative location you used there.
3. Update with your lab subscription id
4. Update $spname, replacing <unique suffix> with 5 random digits
5. In production this script would be run using cloud administrative credentials with authority to create the vault, service principal and secrets. In the lab environment, you will use your existing lab credentials.
6. Run the following command to ensure registration of the KeyVault namespace;

   az provider register --namespace Microsoft.KeyVault

7. Run the script **.\setup-terraform-keyvault.ps1**



Step 2: Developers obtain SP credentials from Key Vault

Examine the PowerShell script **retrieve-sp-credentials.ps1**

Stage 1: Define Parameters and Secure Retrieval Function

In this stage, we define the script parameters and a function to securely retrieve secrets from Azure Key Vault. The script requires the Azure Key Vault name as an input parameter and sets a default file (**sp_credentials.env**) to store the retrieved credentials.

The function **Get-SecretValue** is created to securely fetch secrets stored in Azure Key Vault. Since Azure Key Vault encrypts secrets, PowerShell retrieves them as a SecureString. To convert them into plaintext (for Terraform compatibility), we use **.SecretValue** and decrypt it using **.SecureStringToBSTR().** This ensures Terraform can access credentials without exposing them in plaintext logs.

Stage 2: Authenticate to Azure and Retrieve Credentials

In this stage, the script authenticates to Azure and retrieves the Service Principal credentials stored in Azure Key Vault. The authentication is performed using **Connect-AzAccount**, which ensures the script has access to Azure services.

After authentication, the script fetches the Service Principal Client ID, Client Secret, Tenant ID, and Subscription ID from Azure Key Vault using the **Get-SecretValue** function. These credentials are required for Terraform to authenticate and interact with Azure resources. By securely retrieving these values, we prevent storing static credentials in the script

Stage 3: Set Environment Variables and Store Credentials in a File

Once the credentials are retrieved, they need to be set as environment variables so Terraform can access them during execution. The script assigns the credentials to ARM_CLIENT_ID, ARM_CLIENT_SECRET, ARM_TENANT_ID, and ARM_SUBSCRIPTION_ID using PowerShell's **$env:** syntax. These variables allow Terraform to authenticate automatically without requiring manual input.

Additionally, the credentials are stored in a local .env file, allowing them to persist across sessions or be used in CI/CD pipelines where strict access control is enforced. The .env file format is commonly used to load environment variables dynamically, reducing the need to manually reconfigure Terraform authentication each time.

To ensure security, it is crucial to exclude this file from version control by adding it to .gitignore. This prevents credentials from being accidentally committed to a repository.

Actions

1. Run **az logout** to demonstrate the full process
2. Run the script **.\retrieve-sp-credentials.ps1 -KeyVaultName "<your keyvault>"**
3. You will be prompted for credentials. In production these would be your developer credentials with authority to read from the vault. In the lab environment, use your lab credentials.
4. At this stage, the SP credentials are downloaded and stored and Terraform is now ready to use them. To verify that the credentials have been successfully set, run the following command: **Get-ChildItem Env:ARM_***



5. These environment variables will only last for the duration of the PowerShell session. Prove this by killing your terminal session, opening a new one and repeating the command. No values will be returned.
6. You could re-run the retrieve-sp-credentials script when needed if the secrets where not written out to a local file for security reasons. In our example script they were though, so navigate back to **c:\azure-tf-int\labs\03b** and run **.\load-envs.ps1**. This script reads the env file values back into memory. This allows cross-session credential persistence and is an approach commonly used with CI/CD solutions. If used, access to the env file should be highly restricted.

Verifying the SP account

1. Run **az logout** to ensure you are not using your lab credentials when you interact with azure using Terraform.
2. Verify by running **az logout** again. You should be notified that you have no active accounts

```
There are no active accounts.
```

3. Review the **main.tf** file in your current directory. Update the provider block with your **subscription id**
4. Note that the file is configured to deploy an empty resource group for testing purposes.
5. Run..

   terraform init
   terraform plan
   terraform apply --auto-approve

6. Switch to the portal and verify the existence of resource group RG2

Lab clean-up

1. Run **terraform destroy --auto-approve**
2. Run **./cleanup.ps1** to delete resource group RG1, your Service Principal and Key Vault.
3. Delete local environment variables... **Remove-Item Env:ARM_***

Congratulations, you have completed this lab

©QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026